Assignment No. 03

Generative AI

Fall 2026

Deadline: As Per GCR

Learning to Summarize from Human Feedback
(SFT, reward model, PPO and DPO implemented by you)

Submission Instructions:

- This is a group assignment.
- Each group should be exactly 2 members.
- **Plagiarism is strictly prohibited. Plagiarism among groups would be checked with tools.**
- **Please do not use any AI tool for direct copy / paste and do not copy from others.**
- **AI plagiarism using tools such as ChatGPT, Claude, Gemini, Copilot, Cursor, etc. will be marked straight zero.**
- **Submission carries no marks. Marks will be awarded based on viva.**
- Write a Medium Blog and a LinkedIn Post and submit their links.
- Develop the front end, and push all code to GitHub with commits from both team members. Submit Repo Link.
- Use Colab or Kaggle for GPU access.

Read the whole manual before you begin.

Contents

1 Overview **3**
- 1.1 The problem . . . 3
- 1.2 Constraints . . . 3
- 1.3 Datasets . . . 3

2 Instructions **4**
- 2.1 Task 1 – Run the starter code . . . 4
- 2.2 Task 2 – Stage 1: supervised fine-tuning (SFT) . . . 4
- 2.3 Task 3 – Stage 2: reward model . . . 5
- 2.4 Task 4 – Stage 3: PPO . . . 5
- 2.5 Task 5 – Stage 4: DPO . . . 6
- 2.6 Task 6 – Compare the three policies . . . 7
- 2.7 Task 7 – Analysis . . . 7
- 2.8 Task 8 – Front end and communication . . . 7
- 2.9 Required repository layout . . . 8

3 Evaluation **8**
- 3.1 Automatic metrics . . . 8
- 3.2 Human evaluation . . . 8
- 3.3 Correctness checks . . . 8

4 Results **9**
- 4.1 Table 1 – Data . . . 9
- 4.2 Table 2 – Training . . . 9
- 4.3 Table 3 – Final comparison (300 evaluation posts) . . . 9
- 4.4 Table 4 – Human evaluation (30 posts) . . . 10
- 4.5 Figures . . . 10
- 4.6 Qualitative samples . . . 10

Appendix: Starter Code **11**

1. Overview

1.1 The problem

A pretrained language model can continue text, but it does not know what a *good* summary is. In this assignment you will take a small pretrained model and align it, step by step, with human judgements of summary quality, reproducing the pipeline of *Learning to Summarize from Human Feedback* (Stiennon et al., 2020), the paper that introduced the RLHF recipe later used for ChatGPT.

Input	A Reddit post: subreddit, title and body, followed by `TL;DR:`
Output	A one-sentence summary of the post

You will build one pipeline with four stages, all trained by your own code:

Stage	Model	What it learns
1	SFT policy	to write summaries, by imitating human-written ones
2	Reward model r_ϕ	to score a summary, from pairs of summaries ranked by humans
3	PPO policy	to write summaries the reward model scores highly, without drifting far from SFT
4	DPO policy	the same goal as PPO, directly from the ranked pairs, with no reward model

Stages 3 and 4 both start from the Stage 1 model. At the end you compare the three policies (SFT, PPO, DPO) on posts none of them has seen.

1.2 Constraints

- **One base model for everything:** `HuggingFaceTB/SmolLM2-135M` (135M parameters), fully fine-tuned. It fits every stage on a free Colab or Kaggle T4.
- **You implement the algorithms.** The SFT loss, the reward model and its Bradley–Terry loss, PPO and DPO are written by you in PyTorch, following the lecture notes. No RLHF library.
- **Starter code covers the data side only.** Loading, cleaning, length filtering, tokenisation, batching and model loading are given in the Appendix. Everything from the first loss onwards is your work.
- **Shipped and explained.** The models run behind a small web front end, and all marks come from the viva.

1.3 Datasets

Both datasets were released by OpenAI with the paper and are mirrored on Hugging Face.

Dataset	Content	Used for
`CarperAI/openai_summarize_tldr`	about 130k Reddit posts, each with one human-written summary (`prompt`, `label`)	Stage 1, final evaluation
`CarperAI/openai_summarize_comparisons`	about 260k pairs of summaries of the same post, one preferred by a human labeller (`prompt`, `chosen`, `rejected`)	Stages 2 and 4

SFT data	https://huggingface.co/datasets/CarperAI/openai_summarize_tldr
Preference data	https://huggingface.co/datasets/CarperAI/openai_summarize_comparisons
Base model	https://huggingface.co/HuggingFaceTB/SmolLM2-135M
Paper	https://arxiv.org/abs/2009.01325

The starter code cleans both datasets into one format and keeps fixed random subsets so every group trains on the same data:

Subset	Size	Source split
SFT train / validation	20,000 / 500	tldr `train` / `valid`
Preference train / validation	20,000 / 1,000	comparisons `train` / `valid1`
Evaluation posts	300	tldr `test`

Every prompt ends with `"\nTL;DR:"` and every summary starts with a space and ends with the end-of-sequence token. Posts longer than 448 tokens and summaries longer than 48 tokens are dropped.

Content note

These are real Reddit posts, some from relationship and legal-advice forums, and a few contain mature language. Treat them as research data.

2. Instructions

Complete the tasks in order; each stage uses the checkpoint from an earlier one. Every deliverable must be in your GitHub repository.

Allowed and forbidden

Allowed: `torch`, `transformers` (only `AutoTokenizer`, `AutoModelForCausalLM` and `model.generate`), `datasets`, `rouge-score`, `numpy`, `matplotlib`, the starter code, and any front-end framework.

Forbidden anywhere in your code: `trl`, `peft`, `trlx`, OpenRLHF, DeepSpeed-Chat, the Hugging Face `Trainer` class, `AutoModelForSequenceClassification`, and any other library that implements SFT, reward modelling, PPO or DPO for you.

2.1 Task 1 – Run the starter code

1.1 Set up the environment and run `check_starter.py` (Appendix A.6).

1.2 Report the size of every subset, and the mean and maximum prompt and summary length in tokens for the SFT train subset.

1.3 Write one helper you will use in every stage, `token_logprobs(model, input_ids, attention_mask)`, returning $\log \pi(x_t \mid x_{<t})$ for every position $t \geq 1$, shape $(B, L-1)$, and a `summary_mask` that is 1 exactly on the summary tokens (not prompt, not padding). Check it on one batch by printing the decoded tokens where the mask is 1.

2.2 Task 2 – Stage 1: supervised fine-tuning (SFT)

2.1 Load the base model with `load_causal_lm()`.

2.2 **Loss:** the mean negative log-likelihood of the *summary tokens only*:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{\#\text{summary tokens}} \sum_{t \in \text{summary}} \log \pi_\theta(x_t \mid x_{<t}).$$

Prompt tokens and padding contribute nothing.

2.3 Train for **1 epoch** over the 20,000 SFT pairs: AdamW, learning rate 5×10^{-5}, batch size 16, gradient-norm clipping at 1.0.

2.4 Every 250 steps log the training loss and the validation loss on the 500 SFT validation pairs. Save the final model as `checkpoints/sft/`.

2.5 Generate summaries for 5 evaluation posts (greedy, at most 48 new tokens) before and after SFT, and include them in your report.

2.3 Task 3 – Stage 2: reward model

The reward model takes a post and a summary and returns one number $r_\phi(x, y)$.

3.1 **Architecture.** Load `checkpoints/sft/`. Take the transformer body (`model.model`), drop the language-model head, and add `nn.Linear(hidden_size, 1)`. The reward is this linear layer applied to the final hidden state of the **last non-padding token** of prompt + summary.

3.2 **Loss** (lecture notes, Eq. 40, Bradley–Terry):

$$\mathcal{L}_{\text{reward}}(\phi) = -\,\mathbb{E}_{(x,y_w,y_l)}\Big[\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)\Big].$$

3.3 Train for **1 epoch** over the 20,000 preference pairs: AdamW, learning rate 1×10^{-5}, batch size 8 pairs.

3.4 Report **pairwise accuracy** on the 1,000 validation pairs: the fraction where $r_\phi(x, y_w) > r_\phi(x, y_l)$. Save as `checkpoints/reward/`.

2.4 Task 4 – Stage 3: PPO

Implement PPO exactly as in Sections 2–4 of the lecture notes. Each generated token is an action a_t; the state s_t is the prompt plus all tokens generated so far.

Models in memory:

Model	Initialised from	Trained?
Policy π_θ + value head V_θ	`checkpoints/sft/`, plus a new linear value head on the last hidden layer	yes
Reference π_{ref}	`checkpoints/sft/`	no (frozen)
Reward model r_ϕ	`checkpoints/reward/`	no (frozen)

One PPO iteration:

4.1 **Rollout.** Take 16 prompts from the SFT training subset (prompts only, left-padded with `collate_prompts`). Sample one summary each from π_θ with `do_sample=True`, `temperature=1.0`, `top_k=0`, `top_p=1.0`, at most 48 new tokens. A summary ends at its first end-of-sequence token; ignore everything after it.

4.2 **Store** (no gradients): $\log \pi_{\theta_{\text{old}}}(a_t \mid s_t)$, $\log \pi_{\text{ref}}(a_t \mid s_t)$ and $V_{\theta_{\text{old}}}(s_t)$ for every generated token, and the score $r_\phi(x, y)$ of each prompt + summary. Read $V(s_t)$ from the same position whose logits predict a_t, i.e. the position just before a_t.

4.3 **Per-token reward.** Following the KL-constrained objective (lecture notes, Eq. 41), penalise drifting from the reference at every token and add the reward-model score at the last token T:

$$r_t = -\beta_{\text{KL}}\big(\log \pi_{\theta_{\text{old}}}(a_t \mid s_t) - \log \pi_{\text{ref}}(a_t \mid s_t)\big) \; + \; \mathbf{1}[t = T]\; r_\phi(x, y).$$

4.4 **Returns and advantages** (Eqs. 19–20, 27): with $\gamma = 1$,

$$R_t = \sum_{k \geq 0} \gamma^k r_{t+k}, \qquad A_t = R_t - V_{\theta_{\text{old}}}(s_t).$$

Normalise A_t over all generated tokens in the batch (subtract mean, divide by standard deviation).

4.5 **Update** for $K = 4$ epochs on the same rollout, minimising the full PPO loss (Eqs. 32, 34, 35):

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, \qquad L^{\text{PPO}}(\theta) = \mathbb{E}\Big[\min\big(r_t(\theta)A_t,\; \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\, A_t\big)\Big],$$

$$\mathcal{L}(\theta) = -L^{\text{PPO}}(\theta) + c_V\, \mathbb{E}\big[(V_\theta(s_t) - R_t)^2\big] - c_H\, \mathbb{E}\big[H(\pi_\theta(\cdot \mid s_t))\big].$$

Every expectation is a mean over the generated (non-padding) tokens of the batch. If you run out of GPU memory, process each rollout in two halves and add the gradients before `optimizer.step()`; do not change the settings below.

Use exactly these settings:

Clip ϵ	0.2	KL coefficient β_{KL}	0.05
Value coefficient c_V	0.1	Entropy coefficient c_H	0.01
Discount γ	1.0	PPO epochs K per rollout	4
Optimiser	AdamW, lr 1×10^{-5}	Gradient-norm clipping	1.0
Prompts per rollout	16	Iterations	200 (3,200 prompts)

4.6 After every iteration log: mean r_ϕ score, mean KL to the reference (sum over the summary's tokens of $\log \pi_{\theta_{\text{old}}} - \log \pi_{\text{ref}}$, averaged over the batch), mean summary length in tokens, the **clip fraction** (share of tokens where $r_t(\theta)$ was outside $[1-\epsilon, 1+\epsilon]$), and the three loss terms.

4.7 Save the policy (without the value head) as `checkpoints/ppo/`.

2.5 Task 5 – Stage 4: DPO

5.1 Load `checkpoints/sft/` twice: the trainable policy π_θ and the frozen reference π_{ref}.

5.2 For a sequence, $\log \pi(y \mid x)$ is the **sum** of the token log-probabilities over the summary tokens only.

5.3 **Loss** (lecture notes, Eq. 60), with $\beta = 0.1$:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\,\mathbb{E}_{(x, y_w, y_l)}\Big[\log \sigma\Big(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\Big)\Big].$$

5.4 Train for **1 epoch** over the same 20,000 preference pairs as the reward model: AdamW, learning rate 2×10^{-6}, batch size 8 pairs.

5.5 Every 250 steps log the loss and the **implicit reward accuracy** on the 1,000 validation pairs: the fraction where the expression inside $\sigma(\cdot)$ is positive. Save as `checkpoints/dpo/`.

2.6 Task 6 – Compare the three policies

On the 300 evaluation posts, generate one summary with each of SFT, PPO and DPO (**greedy**, at most 48 new tokens) and compute everything in Section 3. Save all summaries to `results/eval_summaries.jsonl`.

2.7 Task 7 – Analysis

Answer in the blog and in `results/analysis.md`:

7.1 How accurate is your reward model, and what kinds of pairs does it get wrong? Show three.

7.2 During PPO, plot the mean r_ϕ score and the mean KL against iteration. Did the score rise? Did summary length change? Is a higher reward-model score the same as a better summary? Use your human evaluation to answer.

7.3 What role does β_{KL} play? Explain, using Eq. 41, what you would expect to happen with $\beta_{\text{KL}} = 0$. (Reason about it; do not run it.)

7.4 What did the clip fraction look like during training, and what does it tell you about the size of each policy update?

7.5 DPO never trains a reward model. Using Eqs. 55–58, explain where the reward went and why $Z(x)$ does not need to be computed. Compare DPO's implicit reward accuracy with your reward model's accuracy on the same 1,000 validation pairs.

7.6 How many models had to be in GPU memory for PPO, and for DPO? How long did each stage take? Which method would you choose for a new project, and why?

2.8 Task 8 – Front end and communication

8.1 A web interface (Streamlit, Gradio, Flask, ...) where the user pastes a post and sees the SFT, PPO and DPO summaries side by side, each with its reward-model score. It must load *your* checkpoints. Screenshot in the README.

8.2 **Medium blog** (800–1500 words): the RLHF pipeline, your results table, the PPO training curves, and real examples where the methods differ. At least two figures.

8.3 **LinkedIn post** linking the blog and repository.

8.4 **GitHub repository**: public, reproducible README, meaningful commits from *both* members. Upload checkpoints to Hugging Face or Google Drive and link them; do not commit them to GitHub.

2.9 Required repository layout

`starter/`	the given files, unchanged
`common.py`	`token_logprobs`, `summary_mask`, generation helper
`train_sft.py`	Task 2
`train_reward.py`	Task 3 (reward-model class and training)
`train_ppo.py`	Task 4 (value head, rollout, rewards, returns, PPO loss)
`train_dpo.py`	Task 5
`evaluate.py`	Task 6
`results/`	logs, tables, figures, summaries, human evaluation, `analysis.md`
`app/`	front end
`README.md`	how to run everything, links to checkpoints

3. Evaluation

3.1 Automatic metrics

All computed on the 300 evaluation posts, for SFT, PPO and DPO.

Reward-model score	mean $r_\phi(x, y)$ from your frozen reward model
Win rate vs. SFT	share of posts where the method's summary has a higher r_ϕ than the SFT summary of the same post (SFT itself: not applicable)
KL to SFT	mean over posts of $\sum_t \left[\log \pi_\theta(y_t \mid x, y_{<t}) - \log \pi_{\text{ref}}(y_t \mid x, y_{<t})\right]$ on the method's own greedy summary
Length	mean summary length in tokens
ROUGE-L	F-measure against the human reference summary (`rouge-score` package)

3.2 Human evaluation

Take 30 evaluation posts (fixed seed). For each, show the three summaries in random order without saying which model wrote which. Each member independently picks the **best** summary. Report, per member, how many times each method was picked, and the share of posts where the two members agreed. Save the sheet as `results/human_eval.csv`.

What to expect

Chance-level reward-model accuracy is 50%; a 135M model trained on 20,000 pairs will be well short of the large models in the paper, so judge it by whether it is clearly above chance. PPO should raise the mean reward-model score over the course of training while the KL grows slowly; if the KL explodes, check that the reference model is frozen and that the KL penalty has the right sign. Because the KL is estimated from sampled tokens, a batch can show a slightly negative value early in training; that is normal. It is common for PPO to raise the reward-model score while ROUGE-L falls and summaries change length: the policy optimises the reward model, not the human reference. Explain whatever you observe in Task 7.

3.3 Correctness checks

Include the output of these checks in your README.

- **SFT mask:** decoding the tokens where `summary_mask` is 1 gives exactly the summary text followed by the end-of-sequence token.
- **Frozen models:** every parameter of π_{ref} and r_ϕ has `requires_grad = False` during Stages 3 and 4.
- **PPO, first iteration:** the KL to the reference is 0, because the policy still equals SFT.
- **PPO, every iteration:** in the first of the K epochs, $r_t(\theta) = 1$ for every token (up to floating-point error) and the clip fraction is 0.
- **DPO, first step:** the loss equals $\log 2 \approx 0.6931$, because $\pi_\theta = \pi_{\text{ref}}$ makes the argument of σ zero.
- **Padding:** prompts for `generate` are left-padded; the reward model reads the hidden state of the last *real* token, not a padding token. In this tokenizer the padding token *is* the end-of-sequence token, so always find real tokens with `attention_mask`, never with `input_ids != pad_token_id`.

4. Results

Present the following in `results/` and in the blog, using these layouts.

4.1 Table 1 – Data

SFT train / validation pairs	
Preference train / validation pairs	
Evaluation posts	
Mean / max prompt length, SFT train (tokens)	
Mean / max summary length, SFT train (tokens)	

4.2 Table 2 – Training

Stage	**Final train loss**	**Validation metric**	**Time / GPU**
SFT		val. loss:	
Reward model		pairwise acc.:	
PPO		final mean r_ϕ:	
DPO		implicit reward acc.:	

4.3 Table 3 – Final comparison (300 evaluation posts)

Policy	**Mean r_ϕ**	**Win rate vs. SFT**	**KL to SFT**	**Length**	**ROUGE-L**
SFT		–	0		
PPO					
DPO					

4.4 Table 4 – Human evaluation (30 posts)

	SFT picked	PPO picked	DPO picked
Member 1			
Member 2			
Agreement between members		___ / 30	

4.5 Figures

1. SFT training and validation loss.
2. Reward-model training loss and validation accuracy.
3. PPO: mean r_ϕ, mean KL, mean length and clip fraction against iteration.
4. DPO: training loss and validation implicit reward accuracy.
5. Front-end screenshot.

4.6 Qualitative samples

Ten evaluation posts in `results/samples.md` and the blog, each with the human reference and the SFT, PPO and DPO summaries. Include at least two where PPO or DPO is clearly better than SFT and at least two where it is clearly worse, and say why.

Appendix: Starter Code

The starter code covers data loading, cleaning, tokenisation, batching and model loading. Put all files in a folder called `starter/`.

A.1 Environment

```
pip install torch transformers datasets rouge-score matplotlib
```

The model and datasets download automatically from Hugging Face the first time they are used.

A.2 `config.py` – fixed settings

```
MODEL_NAME = "HuggingFaceTB/SmolLM2-135M"   # base model for every stage
SFT_DATA   = "CarperAI/openai_summarize_tldr"
PREF_DATA  = "CarperAI/openai_summarize_comparisons"

MAX_PROMPT_TOKENS  = 448     # post + "TL;DR:" longer than this is dropped
MAX_SUMMARY_TOKENS = 48      # summaries longer than this are dropped

SEED = 42
N_SFT_TRAIN, N_SFT_VAL   = 20_000, 500
N_PREF_TRAIN, N_PREF_VAL = 20_000, 1_000
N_EVAL_POSTS             = 300
```

A.3 `models.py` – tokenizer and model loading

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from config import MODEL_NAME

def load_tokenizer(name=MODEL_NAME):
    tok = AutoTokenizer.from_pretrained(name)
    if tok.pad_token is None:            # SmolLM2 has no pad token
        tok.pad_token = tok.eos_token
    return tok

def load_causal_lm(name_or_path=MODEL_NAME, device=None):
    """Plain pretrained causal LM (no heads added). Use it for the SFT
    policy, the frozen reference model, and as the backbone of the
    reward model and of the PPO value head."""
    device = device or ("cuda" if torch.cuda.is_available() else "cpu")
    model = AutoModelForCausalLM.from_pretrained(name_or_path)
    return model.to(device)
```

A.4 `data.py` – load, clean, filter, tokenise

```
import random
from datasets import load_dataset
from config import *

```

```python
TLDR = "TL;DR:"

def clean_post(raw_prompt):
    """Keep SUBREDDIT/TITLE/POST, drop anything from 'TL;DR:' on,
    and end with one fixed marker the model learns to continue."""
    post = raw_prompt.split(TLDR)[0].strip()
    return post + "\n" + TLDR

def clean_summary(raw):
    """Strip a leading 'TL;DR:', cut stray text after <|endoftext|>,
    and start with exactly one space (it follows 'TL;DR:')."""
    text = raw.split("<|endoftext|>")[0].strip()
    if text.startswith(TLDR):
        text = text[len(TLDR):].strip()
    return " " + text

def _sample_valid(rows, n, tok, make, seed):
    """Shuffle rows with a fixed seed and keep the first n that pass
    the length filter. Each kept item is a dict of token-id lists."""
    order = list(range(len(rows)))
    random.Random(seed).shuffle(order)
    kept = []
    for start in range(0, len(order), 2000):
        for i in order[start:start + 2000]:
            item = make(rows[i], tok)
            if item is not None:
                kept.append(item)
                if len(kept) == n:
                    return kept
    return kept

def _make_sft(row, tok):
    prompt, summary = clean_post(row["prompt"]), clean_summary(row["
        label"])
    p = tok(prompt, add_special_tokens=False)["input_ids"]
    s = tok(summary, add_special_tokens=False)["input_ids"] + [tok.
        eos_token_id]
    if len(summary.strip()) == 0 or len(p) > MAX_PROMPT_TOKENS or len
        (s) > MAX_SUMMARY_TOKENS:
        return None
    return {"prompt_ids": p, "summary_ids": s, "prompt": prompt, "
        summary": summary}

def _make_pref(row, tok):
    prompt = clean_post(row["prompt"])
    ch, rj = clean_summary(row["chosen"]), clean_summary(row["
        rejected"])
    if ch == rj or not ch.strip() or not rj.strip():
        return None
    p  = tok(prompt, add_special_tokens=False)["input_ids"]
    c  = tok(ch, add_special_tokens=False)["input_ids"] + [tok.
        eos_token_id]
    r  = tok(rj, add_special_tokens=False)["input_ids"] + [tok.
        eos_token_id]
    if len(p) > MAX_PROMPT_TOKENS or max(len(c), len(r)) >
        MAX_SUMMARY_TOKENS:
        return None
    return {"prompt_ids": p, "chosen_ids": c, "rejected_ids": r,
```

```
            "prompt": prompt, "chosen": ch, "rejected": rj}

def load_sft_splits(tok):
    ds = load_dataset(SFT_DATA)
    train = _sample_valid(ds["train"], N_SFT_TRAIN, tok, _make_sft,
        SEED)
    val   = _sample_valid(ds["valid"], N_SFT_VAL,   tok, _make_sft,
        SEED)
    return train, val

def load_preference_splits(tok):
    ds = load_dataset(PREF_DATA)
    train = _sample_valid(ds["train"],  N_PREF_TRAIN, tok, _make_pref
        , SEED)
    val   = _sample_valid(ds["valid1"], N_PREF_VAL,   tok, _make_pref
        , SEED)
    return train, val

def load_eval_posts(tok):
    """Held-out posts (with their human reference summary) used to
    compare SFT, PPO and DPO at the end. Never train on these."""
    ds = load_dataset(SFT_DATA)
    return _sample_valid(ds["test"], N_EVAL_POSTS, tok, _make_sft,
        SEED)
```

A.5 batching.py – padded batches

```
import torch

def _pad(seqs, pad_id, side):
    L = max(len(s) for s in seqs)
    ids  = torch.full((len(seqs), L), pad_id, dtype=torch.long)
    mask = torch.zeros((len(seqs), L), dtype=torch.long)
    for i, s in enumerate(seqs):
        t = torch.tensor(s, dtype=torch.long)
        if side == "right":
            ids[i, :len(s)], mask[i, :len(s)] = t, 1
        else:                                       # left padding
            ids[i, L - len(s):], mask[i, L - len(s):] = t, 1
    return ids, mask

def collate_sft(batch, pad_id):
    """Right-padded prompt+summary. prompt_lens[i] = number of prompt
    tokens in row i, so summary tokens are positions prompt_lens[i]
        .. end."""
    seqs = [b["prompt_ids"] + b["summary_ids"] for b in batch]
    ids, mask = _pad(seqs, pad_id, "right")
    prompt_lens = torch.tensor([len(b["prompt_ids"]) for b in batch])
    return {"input_ids": ids, "attention_mask": mask, "prompt_lens":
        prompt_lens}

def collate_pref(batch, pad_id):
    """Two right-padded batches (prompt+chosen, prompt+rejected) that
    share the same prompt_lens."""
    ch, ch_mask = _pad([b["prompt_ids"] + b["chosen_ids"]   for b in
        batch], pad_id, "right")
    rj, rj_mask = _pad([b["prompt_ids"] + b["rejected_ids"] for b in
        batch], pad_id, "right")
```

```
    prompt_lens = torch.tensor([len(b["prompt_ids"]) for b in batch])
    return {"chosen_ids": ch, "chosen_mask": ch_mask,
            "rejected_ids": rj, "rejected_mask": rj_mask,
            "prompt_lens": prompt_lens}

def collate_prompts(batch, pad_id):
    """LEFT-padded prompts only, for model.generate() in PPO rollouts
    and evaluation. Left padding keeps every prompt's last token at the
    same position, which is where generation starts."""
    ids, mask = _pad([b["prompt_ids"] for b in batch], pad_id, "left")
    return {"input_ids": ids, "attention_mask": mask}
```

A.6 check_starter.py – where your code begins

```
from functools import partial
from torch.utils.data import DataLoader
from models import load_tokenizer
from data import load_sft_splits, load_preference_splits, load_eval_posts
from batching import collate_sft, collate_pref, collate_prompts

tok = load_tokenizer()
sft_train, sft_val   = load_sft_splits(tok)
pref_train, pref_val = load_preference_splits(tok)
eval_posts           = load_eval_posts(tok)
print("SFT  train/val :", len(sft_train), len(sft_val))
print("Pref train/val :", len(pref_train), len(pref_val))
print("Eval posts     :", len(eval_posts))

ex = sft_train[0]
print("\n--- one SFT example ---")
print(ex["prompt"][-300:])
print("SUMMARY:", repr(ex["summary"]))

pad = tok.pad_token_id
b = next(iter(DataLoader(sft_train,  8, shuffle=True, collate_fn=partial(collate_sft,  pad_id=pad))))
print("\nSFT batch      :", tuple(b["input_ids"].shape), "prompt_lens", b["prompt_lens"].tolist())
b = next(iter(DataLoader(pref_train, 8, shuffle=True, collate_fn=partial(collate_pref, pad_id=pad))))
print("Pref batch     :", tuple(b["chosen_ids"].shape), tuple(b["rejected_ids"].shape))
b = next(iter(DataLoader(eval_posts, 8, collate_fn=partial(collate_prompts, pad_id=pad))))
print("Prompt batch   :", tuple(b["input_ids"].shape), "(left-padded)")
```

Use these collate functions with `torch.utils.data.DataLoader`, for example `DataLoader(sft_train, batch_size=16, shuffle=True, collate_fn=partial(collate_sft, pad_id=tok.pad_token_id))`.

Good luck. A model is only as aligned as the reward it is given.